DeFuzz: Invariant-Guided Agentic Testing of Compiler-Instrumented Defenses

Jingyao Zhang
School of Cyber Science and Engineering
Southeast University
Nanjing, China
zhangjingyao@seu.edu.cn

Abstract— Compiler-instrumented defenses, such as stack canaries, Control-Flow Integrity (CFI), and `_FORTIFY_SOURCE`, protect software against memory errors and control-flow hijacking. However, these defenses can fail silently—the compiler produces a working binary without any warnings or crashes, but the security protection is quietly weakened or broken. For example, in CVE-2023-4039, dynamic stack allocation allowed an overflow to bypass the GCC stack canary on AArch64. Such failures expose a fundamental oracle gap: conventional crash and differential testing only check whether a program produces correct outputs, making them completely blind when compilers silently weaken or break a defense.

In this paper, we present DEFUZZ, an invariant-directed agentic framework for discovering silent failures in compiler-instrumented defenses. DEFUZZ first extracts formal security invariants from compiler source code, Instruction Set Architecture (ISA) specifications, and historical bug fixes using a hybrid CoT and RAG pipeline. It then maps these invariants to compiler passes and decision branches, locating where defenses are implemented and controlled. To detect violations, DEFUZZ turns this mapped logic into programmatic bug oracles, implemented as static and dynamic checkers across target ISAs. All bug verdicts are decided strictly by these checkers, with zero LLM involvement in bug adjudication. Guided by the mapped compiler locations, specialized LLM agents perform directed fuzzing to hit targeted defense branches, synthesizing and mutating seeds to trigger invariant violations under oracle evaluation. We evaluated DEFUZZ on GCC and LLVM across multiple ISAs. DEFUZZ discovered [TBD: N] previously unknown silent failures in compiler-instrumented defenses, of which [TBD: M] have been confirmed upstream. Some of these bugs were rated as high-severity vulnerabilities. These results show that DEFUZZ effectively uncovers critical silent defense bugs that evade existing compiler testing techniques.

I. INTRODUCTION

Compilers deploy a wide range of compiler-instrumented defenses to harden software against memory corruption, control-flow hijacking, and sensitive data leakage. During compilation, these defenses insert runtime bounds checks, enforce stack layout rules, or clear registers to block exploit primitives. However, implementing these defenses accurately inside production compilers like GCC and LLVM is difficult. Defense code must interact with complex optimization passes and translate abstract protection rules into machine instructions across diverse target ISAs. This translation relies on strict assumptions about object bounds, instruction semantics, register conventions, and stack frame layouts. When the implementation of a defense mechanism fails to match these ISA or optimization assumptions, its security protection breaks from within.

When a defense implementation goes wrong, it often causes **silent defense failures**: the compiler produces a working binary without any warnings or crashes, but the security protection is quietly weakened or broken. Because the compiled program still runs normally on benign inputs, standard functional tests pass completely, leaving the software exposed to attack. For example, in **CVE-2023-4039**, flawed frame layout logic in GCC's stack protector on AArch64 placed dynamically allocated buffers outside the canary-protected region. This flaw allowed a buffer overflow to overwrite the saved return address without touching the stack canary. We examine the root cause and stack layout mechanics of this bug in Section II.

Existing compiler testing techniques fail to catch such silent bugs due to two fundamental barriers. The first barrier is the **oracle gap**. Widely used compiler fuzzers (e.g., Csmith [1] and YARPGen [2]) test functional correctness by checking for compiler crashes or output mismatches across optimization levels. Because a binary with a broken defense still produces correct outputs on normal inputs, crash and differential oracles are completely blind when a defense is weakened or broken. The second barrier is reaching deep defense decision branches. Triggering silent defense bugs requires specific code patterns—such as pairing a `__strcat_chk` call on an opaque-length buffer with a subsequent `strlen` call to trigger `_FORTIFY_SOURCE` bound relaxation in GCC's `strlen` pass—that random mutators rarely generate. Meanwhile, static analyzers struggle to model defense logic across multiple ISAs, and unconstrained LLM agents suffer from severe hallucinations and false bug reports.

To address these barriers, we present **DEFUZZ**, an invariant-guided agentic testing framework for discovering silent failures in compiler-instrumented defenses. Our core insight is that while compiler defense implementations vary across backends, their underlying security contracts can be expressed as target-independent *security invariants*. By converting these invariants into programmatic checkers, we directly verify whether compiled code preserves intended protections, closing the silent defense oracle gap. To generate inputs that reach deep defense branches, DEFUZZ combines invariant constraints

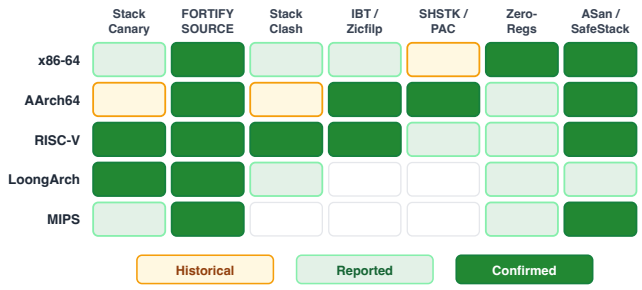


Fig. 1. Partial matrix of silent compiler defense bugs across representative defense mechanisms and target ISAs.

with call-graph-guided agentic directed fuzzing to locate and hit target defense logic in the compiler, while leaving all bug verdicts to deterministic checkers. This design overcomes the blind spots of random mutators without suffering from model hallucination.

Building DEFUZZ requires solving three main challenges. First, we need to obtain reliable security invariants. Because compilers themselves have bugs, we cannot trust their current outputs as the ground truth. Instead, DEFUZZ extracts invariants from compiler source code, ISA specifications, and historical bug fixes using a hybrid CoT and RAG pipeline. Second, we need to check these invariants accurately across different ISAs without false positives. Each ISA uses different registers, instructions, and stack layouts, and asking an LLM to judge compiled code directly causes hallucinations. DEFUZZ solves this by turning each invariant into ISA-specific static and dynamic checkers, with **zero LLM involvement in bug adjudication**. Third, we need to generate test programs that actually reach target defense logic in the compiler. Checkers cannot catch a bug if test inputs never trigger the compiler branches that control a defense. DEFUZZ solves this by mapping each invariant to its compiler passes and decision branches via call-graph analysis. Guided by these mapped locations, LLM agents use execution feedback to generate and mutate seeds until those branches are hit.

We evaluated DEFUZZ on GCC and LLVM across multiple ISAs. DEFUZZ discovered **[TBD: N] previously unknown silent failures in compiler-instrumented defenses**, of which **[TBD: M] have already been confirmed upstream** (Figure 1). Some of these bugs were rated as high-severity vulnerabilities. These results show that DEFUZZ effectively uncovers critical silent defense bugs that evade existing compiler testing techniques.

In summary, this paper makes the following contributions:

- **Automated Invariant Extraction:** We extract formal security invariants from compiler source code, ISA specifications, and historical bug fixes using a hybrid CoT and RAG pipeline, capturing the protection rules of compiler defenses without relying on existing compiler outputs.
- **Multi-ISA Programmatic Oracles:** We turn security invariants into static and dynamic checkers across target ISAs to verify compiled code with zero LLM involvement in bug adjudication, closing the silent defense oracle gap.
- **Call-Graph-Guided Agentic Fuzzing:** We map invariants to compiler passes and decision branches via call-graph analysis, and guide specialized LLM agents with execution feedback to generate and mutate seeds that hit target defense logic.
- **New Silent Defense Bugs:** We evaluate DEFUZZ on GCC and LLVM across multiple ISAs and discover **[TBD: N]** previously unknown silent failures in compiler-instrumented defenses, with **[TBD: M]** confirmed upstream.

II. BACKGROUND AND MOTIVATION

A. Compiler-Instrumented Defenses

A compiler-instrumented defense introduces checks, layout constraints, or metadata at compile time to enforce a security policy at run time. Enforcement may also require runtime libraries, operating-system support, or hardware features. Stack canaries use guard comparisons, CFI restricts indirect control-flow targets, and PAC uses signing and authentication instructions. The presence of such instrumentation alone does not establish that the intended protection holds.

Defense logic can span target-independent IR and backend representations such as RTL, SelectionDAG, or Machine IR. Lowering must respect instruction semantics and ABI conventions for registers, stack alignment, and call frames. For instance, dynamic allocations through `alloca` or VLAs require target-specific stack-pointer adjustments and can change the relation between buffers, saved registers, and guards. A property preserved by one backend may therefore fail in another.

B. Sources of Security Invariants

Security invariants express required protection, rather than merely describing behavior observed in an existing binary. Their evidence comes from three sources: compiler source and documentation, ISA and ABI specifications, and historical bug reports with their fixes. Documentation and specifications describe intended constraints; source comments and assertions identify where they apply; bug reports and patches expose assumptions that an implementation failed to preserve. Because the implementation under test can be wrong, its current behavior is not by itself a specification.

Recent work also separates property generation from verification. PropertyGPT [3] retrieves human-written smart-contract properties to generate new candidates and checks them with a dedicated prover. CIll [4] uses model-checking feedback to refine LLM-generated inductive invariants. These approaches motivate pairing generated properties with independent checking. Our task differs from proving a program assertion: we extract the security requirements of compiler defenses and search for compiled artifacts that violate them. An invariant must state its mechanism, applicability conditions, required property, and supporting evidence before it can guide a checker and a test seed.

C. Threat Model and Scope

We evaluate emitted defense code under the assumptions of each defense mechanism:

- **Assumptions:** The application contains a memory-safety vulnerability that an attacker can trigger. The defense is enabled and its required runtime and hardware support is present. Attacker capabilities are limited to the threat model of the mechanism being tested; a stack canary, for example, does not promise to stop arbitrary writes that skip the guard.
- **In-Scope:** We study silent failures in compiler-instrumented defenses: compilation and ordinary functional tests succeed, but the emitted code violates a security invariant under the documented preconditions. Tests may inspect binary structure or exercise a violation at run time. They establish the failed protection without requiring an end-to-end exploit.
- **Out-of-Scope:** We do not search for application-level vulnerabilities or intentional source-level bypasses such as inline assembly that disables protection. Hardware vulnerabilities, side-channel attacks, and business logic flaws are also excluded. We evaluate standard defense emissions and exclude interactions between compilation optimization options and defense mechanisms.

The source corpus used for invariant extraction is pinned to GCC 16.1; bug-testing results are tied to the compiler version and target configuration recorded for each run.

D. Silent Failure: A Representative Case

CVE-2023-4039 illustrates the connection between a defense requirement, its architectural assumptions, and a testable violation [5]. For a contiguous stack overflow covered by the stack-protector policy, the guard must lie on the overwrite path to the saved return address. In the affected AArch64 layout, the saved return address lies at a higher address than the VLA or `alloca` buffer but below the canary (Figure 2). An overflow toward higher addresses can therefore corrupt the return address without changing the guard. The canary comparison still executes and passes.

The report traces this failure to a mismatch between target-independent guard placement and the AArch64 frame layout. It suggests both a seed containing a dynamic allocation and a checker that resolves the relative locations of the allocation, guard, and saved return address. Merely finding a canary comparison in the binary would miss the failure.

A separate failure, GCC PR-96191 [6], concerns a canary value left in a temporary register after the AArch64 stack-protector check. The report cites the machine-description requirement that intermediate values must not survive the protection sequence. This yields a different invariant, about register state rather than stack layout, and requires a checker that follows the relevant register updates. The two cases show why the security property and its ISA-specific observation must both be explicit.

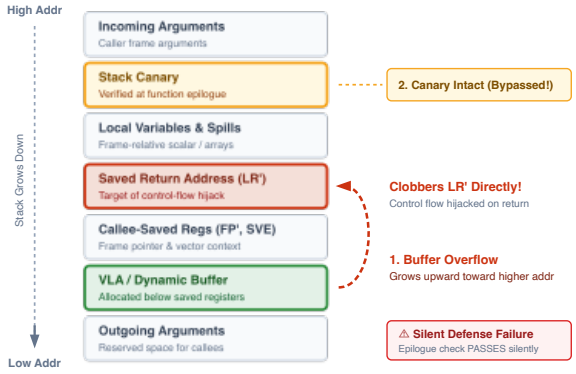


Fig. 2. CVE-2023-4039: an overflow from the VLA reaches the saved return address before the canary, leaving the guard unchanged.

E. Why Existing Methods Miss It

Crash and differential testing can reveal functional miscompilations, but matching outputs on non-attacking inputs do not validate a defense. Moreover, attack-triggering executions may involve source-level undefined behavior, making ordinary cross-compiler output agreement unsuitable as the security oracle. Tests must instead check the protection promised by the enabled defense.

Static checking can detect a known violation when its rule models the relevant semantics. A symbol or instruction-presence rule, however, cannot establish correct guard placement or register cleanup. Extending such checks requires defense-specific properties and target-specific interpretation.

Generic seed generation does not directly connect those properties to the program structures that exercise them. Random generation may reach such structures, but coverage alone does not say whether a seed tests the assigned invariant. This motivates agents that inspect source context and revise seeds from checker feedback. Their role is to direct the search; executable checkers remain responsible for deciding whether the generated artifact violates the defense requirement.

III. Design of DeFuzz

In this section, we present the design of DeFuzz, an invariant-directed framework for discovering silent failures in compiler-instrumented defenses. Figure 3 illustrates its overall workflow, which consists of four core steps: invariant generation, compiler source mapping, programmatic oracle construction, and agentic directed fuzzing. First, we extract formal security invariants from compiler corpora and historical defects using a hybrid CoT and RAG pipeline. With these invariants established, we map them to the relevant compiler passes and decision branches, identifying where defenses are implemented and which conditions govern their execution. Next, we turn this mapped logic into programmatic bug oracles. We implement these oracles as static and dynamic checkers across target ISAs to determine whether program seeds expose defense failures, mitigating false positives from

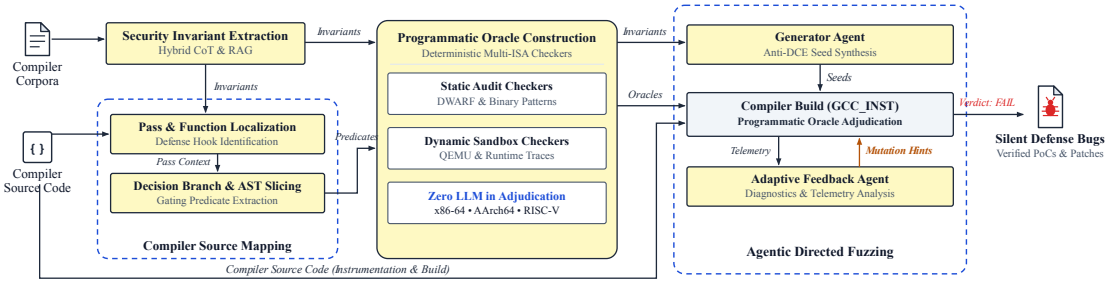


Fig. 3. Overview of DEFUZZ. The framework operates across four core steps: (1) extracting security invariants from compiler corpora and historical defects via a hybrid CoT and RAG pipeline; (2) mapping invariant logic to compiler passes and decision branches; (3) constructing programmatic static and dynamic oracles across target ISAs; and (4) performing agentic directed fuzzing, where a generator agent synthesizes probe seeds and a feedback agent steers iterative exploration under programmatic oracle verification.

model hallucination. Finally, our agentic directed fuzzing coordinates multiple specialized agents: a generator agent synthesizes program seeds designed to violate target invariants, while a feedback agent analyzes execution feedback to guide iterative mutations, with our programmatic oracles evaluating and verifying each candidate. We discuss each step in detail in the following subsections.

A. Security Invariant Generation

The first step extracts a catalog of verifiable security invariants from compiler source code, architecture manuals, and historical bug patches.

1) Formal Invariant Contract: A *security invariant* is a rule that a compiler-instrumented defense must satisfy under its threat model to protect a program. When compiled code violates this rule, the defense is silently weakened or broken, even if the binary compiles without warnings and passes functional tests.

To support automated checking across backends, DeFuzz represents each security invariant ϕ as a six-part tuple:

$$\phi = \langle \mathcal{M}, \mathcal{A}, \mathcal{P}_{\text{cond}}, \mathcal{S}_{\text{inv}}, \mathcal{O}_{\text{phenom}}, \mathcal{E}_{\text{prov}} \rangle \tag{1}$$

Here, $\mathcal{M}$ is the target defense mechanism (e.g., stack canary, CFI, or `_FORTIFY_SOURCE`), $\mathcal{A}$ lists the applicable ISAs, $\mathcal{P}_{\text{cond}}$ defines the source-level activation preconditions (e.g., dynamic stack allocation via VLA or `alloca`), $\mathcal{S}_{\text{inv}}$ states the invariant rule, $\mathcal{O}_{\text{phenom}}$ describes the observable violation signal, and $\mathcal{E}_{\text{prov}}$ stores the supporting source evidence.

DeFuzz separates the abstract violation signal $\mathcal{O}_{\text{phenom}}$ from the concrete checking code. For instance, "the stack canary does not sit between local buffers and the saved return address" is an abstract violation signal. Checking stack offsets in DWARF debug records or assembly output is a concrete detection recipe implemented later by the oracle (§III-C).

2) Candidate Extraction via Segmented CoT: To avoid circular self-validation from buggy compilers, DeFuzz mines invariants from three trusted sources: compiler source comments and assertions, official ISA and ABI manuals, and historical CVE reports with upstream fix patches.

In production compilers, the implementation of a single defense is spread across target-independent middle-end passes and target-specific backend files. Feeding millions of lines of compiler source code into an LLM at once causes severe context loss and hallucination. Therefore, DeFuzz splits the compiler source tree into focused partitions along pass and function boundaries. It then uses a segmented Chain-of-Thought (CoT) prompt [7] to analyze each partition independently. For each code block, the model identifies the protected target, the activation condition, and the intended security guarantee. Every candidate invariant must cite exact source files and line numbers as provenance evidence $\mathcal{E}_{\text{prov}}$.

3) Cross-Mechanism Transfer via Hybrid RAG: Segmented CoT extracts rules within individual defenses, but different defenses often share the same underlying failure modes. To find similar flaws across defenses and backends, DeFuzz uses a Retrieval-Augmented Generation (RAG) [8] pipeline based on abstract failure transfer.

Standard code search matches specific variable or function names, which traps the search inside a single defense mechanism. Instead, DeFuzz generalizes confirmed historical bugs into mechanism-independent failure patterns through four steps:

1) **Distillation:** Summarize a confirmed bug into a general failure pattern. For example, CVE-2023-4039 is generalized from "AArch64 VLA overflows return address before canary" into "dynamic stack allocations bypass statically calculated guard boundaries."
2) **Analogy Filtering:** Scan candidate code blocks across other defenses (such as SafeStack, ShadowCallStack, or PAC) and discard superficial keyword matches that do not share the same structural logic.
3) **Specialization:** Map the general failure pattern onto the target defense's data structures, such as shadow stack pointers or PAC signing instructions.
4) **Entailment Check:** Verify that the specialized invariant rule logically follows from the retrieved source code and comments.

a) Dual-Aspect Query Formulation: To search the compiler codebase effectively, DeFuzz encodes each failure pattern into two query forms: a dense semantic summary (`root_cause_phrase`) and sparse code tokens

TABLE I
TRANSFERRED INVARIANTS BY RETRIEVAL BACKEND (24 PROBES, 4,496 GCC-16.1 CHUNKS).

Category	Count	Note
Intersection (BM25 ∩ embed)	5	Same seed & chunk, isomorphic statement
BM25-only	4	Embedding missed the block for that seed
Embedding-only	2	BM25 missed; dense retrieval added
Union (deduplicated)	**11**	$9+7-5$; all verified as novel

(`critical_tokens`, such as register macros and pass flags).

We tested sparse BM25 and dense embedding retrieval using 24 historical bug probes across 4,496 GCC-16.1 source chunks. BM25 captures exact identifier matches (e.g., `ix86_zero_call_used_regs`, `SUBREG_PROMOTED`), while dense retrieval finds similar defense logic written with different variable names across backends (e.g., ARM CMSE register clearing and RISC-V saturating truncations). After deduplicating by seed, code chunk, and invariant statement, the two retrievers find 5 shared invariants, 4 BM25-only invariants, and 2 embedding-only invariants, yielding 11 novel invariants in total (Table I).

4) Grounding Gates and Catalog Maintenance: Before entering the final catalog Φ, every candidate invariant from CoT or RAG must pass three filtering gates:

- **Gate 1 (Evidence Check):** Confirms that the cited source files, AST structures, or architecture manuals directly support the stated precondition and security rule.
- **Gate 2 (Observability Check):** Verifies that the violation signal $\mathcal{O}_{\text{phenom}}$ can be observed deterministically in compiled binaries or runtime traces.
- **Gate 3 (Deduplication):** Computes cosine similarity against existing invariants in Φ and drops candidates above an 85.0% threshold to avoid duplicate testing.

Validated invariants are stored in the catalog $\Phi = \{\phi_1, \phi_2, \ldots, \phi_n\}$. Whenever online fuzzing discovers a new silent defense bug, DeFuzz distills its root cause and feeds it back into the RAG probe pool to uncover related bugs in future runs.

B. Compiler Source Mapping

With the security invariants established, DeFuzz maps each invariant to the compiler passes and decision branches that control its implementation.

1) MCP-Guided Codebase Navigation: Production compilers such as GCC and LLVM contain millions of lines of C and C++ code. Finding the exact files and conditional branches that govern a security defense is difficult for automated tools. To search this codebase accurately, DeFuzz equips a coding agent with read-only code analysis tools via the Model Context Protocol (MCP).

The MCP toolkit provides four analysis tools: `call-graph` to trace function calls across optimization passes, `ast-parser` to inspect C/C++ syntax trees, `alia-ast` to examine intermediate representations, and `ripgrep` for fast symbol and macro lookup. Using these tools, the agent traces how each defense is lowered from middle-end passes into backend machine instructions.

2) Three-Level Decision Hierarchy: Through MCP-guided code navigation, the agent maps each security invariant ϕ to a three-level hierarchy in the compiler source tree:

- **Level 1 (Compiler Pass):** Locates the middle-end or backend pass that implements the defense (e.g., `pass_stack_protect` in GCC or `StackProtectorPass` in LLVM).
- **Level 2 (Target Functions):** Pinpoints the functions that calculate stack layouts, insert guard checks, or lower instructions (e.g., `expand_used_vars()` in GCC or `LayoutStackNodes()` in LLVM).
- **Level 3 (Critical Structs and Branches):** Identifies the exact data structures and conditional branches that control defense placement, such as `has_vla`, `stack_protect_decl`, and `canary_loc`.

This three-level mapping links abstract invariant preconditions directly to concrete decision branches in the compiler source code.

3) Compile-Time Branch Probing (`GCC_INST`): Once the Level-3 decision branches are mapped, the coding agent instruments the compiler source code by inserting lightweight logging probes:

```
DEFUZZ_LOG("SP#a3f2:hit", slot);
```

Here, `"SP#a3f2"` is a unique identifier (UID) that encodes the defense mechanism, source file, function, and branch index. These probes run during compilation passes, recording which defense branches were taken and logging internal values such as stack slot offsets. Because the probes only record compiler-internal states at build time, they never alter the generated machine code or add runtime overhead to the emitted target binary.

After inserting the probes, DeFuzz rebuilds the compiler into an instrumented driver, denoted `GCC_INST` (or `CLANG_INST` for LLVM). Whenever `GCC_INST` compiles a test seed, it writes a deterministic UID trace to disk, showing the fuzzing loop whether targeted defense branches were exercised.

C. Programmatic Oracle Construction

Next, DeFuzz turns the mapped invariant logic into programmatic bug oracles across target ISAs, eliminating false positives from model hallucination (Figure 4).

1) Oracle Contract and Four-State Verdict Matrix: DeFuzz defines each bug oracle as a deterministic contract. Let $\mathcal{P}$ denote the space of compiled program binaries, $\mathcal{I} = \{\text{x86-64}, \text{AArch64}, \text{RISC-V}\}$ be the set of target ISAs, and

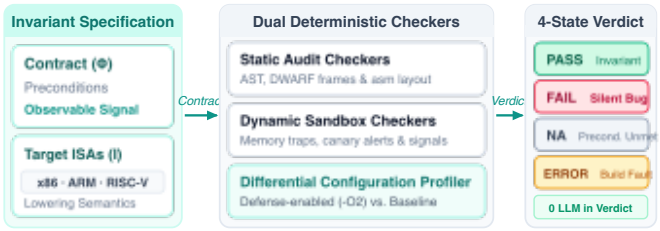


Fig. 4. Multi-ISA deterministic oracle architecture. Invariant contracts are specialized across target ISAs and instantiated into dual static audit and dynamic sandbox checkers, yielding a deterministic four-state verdict without LLM adjudication.

$\mathcal{S} \subseteq \Phi$ be the set of security invariants. The oracle operates as a deterministic function:

$$\mathcal{O} : \mathcal{P} \times \mathcal{I} \times \mathcal{S} \longrightarrow \{\text{PASS}, \text{FAIL}, \text{NA}, \text{ERROR}\} \quad (2)$$

Each output state carries a clear meaning:

- **PASS:** The compiled binary satisfies the security invariant on the target architecture.
- **FAIL:** The binary violates the security invariant, confirming a silent compiler defense bug.
- **NA (Not Applicable):** The target defense was disabled, or the test seed did not trigger the invariant's precondition.
- **ERROR:** Compilation failed due to syntax errors or environment faults. The system logs this state as a build diagnostic and never reports it as a defense bug.

LLM agents never adjudicate or override these verdicts. Every bug report requires deterministic verification by the oracle function $\mathcal{O}$, ensuring zero LLM involvement in bug adjudication.

2) Dual Static and Dynamic Checkers: Compiler defense failures manifest in two distinct forms. Some failures leave structural defects in emitted binary or assembly code, which can be observed without executing the program. Other failures only emerge during runtime execution, where compiler optimizations silently bypass protection boundaries. To catch both classes of bugs deterministically, DeFuzz builds two complementary types of programmatic checkers. Static checkers inspect unexecuted compiler artifacts, while dynamic checkers monitor runtime execution traces. All bug verdicts are computed directly by these checkers, ensuring zero LLM involvement in bug adjudication.

a) Static Checkers (Binary and Assembly Audit): Static checkers inspect unexecuted compiler outputs, including assembly listings, machine instructions, stack frame layouts, and ELF symbol tables. They verify structural defense properties without executing the test binary.

Consider stack canary protection in GCC as an example. When checking a canary at function return, the compiler loads the reference canary into a scratch register and compares it against the stack guard slot. In vulnerable compiler versions, the compiler forgot to clear this scratch register before returning to the caller. This flaw allowed the secret canary value to leak through caller-accessible registers, enabling attackers to read the canary and bypass stack protection entirely.

Listing 1. Static/inline checker for canary register leakage.

```
// Scan registers right after return
void audit_canary_leak(uintptr_t canary) {
  uintptr_t r0, r1, r2 /* ... */;
  asm volatile(
    "mov␣%%rax,␣%0;␣mov␣%%rbx,␣%1;␣mov␣%%rcx,␣%2\n\t"
    // ... (for all target ISA registers)
    : "=r"(r0), "=r"(r1), "=r"(r2)
  );
  // FAIL if any register retains canary
  if (r0 == canary || r1 == canary /* ... */)
    emit_verdict(FAIL, "Canary␣leaked");
  else
    emit_verdict(PASS, "Canary␣cleared");
}
```

Listing 1 shows how DeFuzz detects this flaw. The checker audits the emitted assembly and executes an inline probe right after the epilogue, scanning all general-purpose registers for the canary secret. If any register still holds the canary value, the checker records a FAIL verdict. Otherwise, it records a PASS.

b) Dynamic Checkers (Runtime Sandbox Verification): Dynamic checkers execute compiled test binaries inside an isolated sandbox with dedicated signal handlers and memory fault traps. They evaluate runtime security guarantees that static layout analysis cannot reliably verify.

As an example, consider `_FORTIFY_SOURCE` in GCC (PR 125079). Under this defense, `strcat` is rewritten into the bounds-checked `__strcat_chk(dst, src, objsize)`. When `dst` has an unknown initial string length and is later passed to `strlen(dst)`, GCC's `tree-ssa-strlen` pass (`get_string_length`) optimizes `__strcat_chk` into `__stpcpy_chk(dst + strlen(dst), src, objsize)`. Although GCC advances the destination pointer by `strlen(dst)` bytes, it forgets to subtract `strlen(dst)` from `objsize`. As a result, the runtime check still allows writing a full `objsize` bytes starting from the shifted pointer, silently permitting an overflow of up to `strlen(dst)` bytes past the end of `dst`.

Listing 2 shows the dynamic verification harness for this invariant. After initializing a 64-byte buffer with an 8-byte prefix via an opaque helper, the test appends a 60-byte string via `__strcat_chk` followed by `strlen`. Because $8 + 60$ exceeds the 64-byte buffer, a working defense must abort with `SIGABRT`, which the harness catches to record PASS. Because GCC leaves the bound at 64 instead of $64 - 8 = 56$, the 60-byte write finishes without aborting and reaches the failure sentinel, flagging a deterministic FAIL.

Listing 2. Dynamic checker for _FORTIFY_SOURCE bounds.

```
void test_fortify_strcat(const char *src60) {
  char buf[64];
  init_opaque_8bytes(buf); // strlen(buf)==8,
       unknown to pass
  setup_sigabrt_handler(); // Trap on fortify abort

  // Rewritten to __stpcpy_chk(buf + 8, src60, 64)
  // Missing bound subtraction allows 8-byte
       overflow
  __builtin___strcat_chk(buf, src60, sizeof(buf));
  sink = __builtin_strlen(buf); // Triggers strlen
       fold

  emit_verdict(FAIL, "Overflow␣unchecked");
}

void sigabrt_handler(int sig) {
  emit_verdict(PASS, "Overflow␣trapped");
  _exit(0);
}
```

c) Redundant Cross-Validation: When an invariant can be observed both statically and dynamically, DeFuzz generates dual checkers to cross-validate results. For example, the static checker inspects the compiler IR and assembly to verify that `__stpcpy_chk` receives a decremented size argument (`objsize - strlen(dst)`). Meanwhile, the dynamic checker confirms that an overflowing append triggers a runtime abort. This dual design prevents false negatives where the compiler emits defense calls with stale or permissive bounds. Every reported bug represents a deterministic, reproducible compiler defense failure without human or LLM bias.

3) Target-Specific Specialization Across ISAs: A security invariant defines a universal defense rule, but its machine-level enforcement depends on target architecture semantics. Different ISAs use distinct instruction sets, register conventions, and stack frame layouts. Therefore, for each invariant $s \in \mathcal{S}$ and target ISA $i \in \mathcal{I}$, DeFuzz generates a dedicated checker $C_{s,i}$.

Consider stack canary protection across architectures. On x86-64, the `call` instruction pushes the return address directly onto the stack above the saved frame pointer. On AArch64 and RISC-V, the CPU holds the return address in a link register (`x30` or `ra`), spilling it to the stack only in non-leaf functions. Thus, an x86-64 layout checker verifies buffer offsets relative to the frame pointer. In contrast, an AArch64 checker tracks whether dynamic allocations place buffers above the saved link register slot. Similarly, register leak checkers (Listing 1) scan `rax–r15` on x86-64, `x0–x30` on AArch64, and `x1–x31` on RISC-V. This target-specific specialization prevents valid architectural differences from triggering false bug reports.

4) Differential Configuration Profiling: To isolate defense-specific behaviors, each checker runs the same test seed under two compilation profiles: a defense-enabled build (e.g., `-fstack-protector-strong`) and an unprotected baseline build. Comparing both outputs confirms that observed layout changes or runtime traps come from the target defense rather than unrelated optimization passes. When a checker confirms a FAIL verdict under the defense-enabled build, the oracle emits a bug alert with full binary and execution trace evidence.

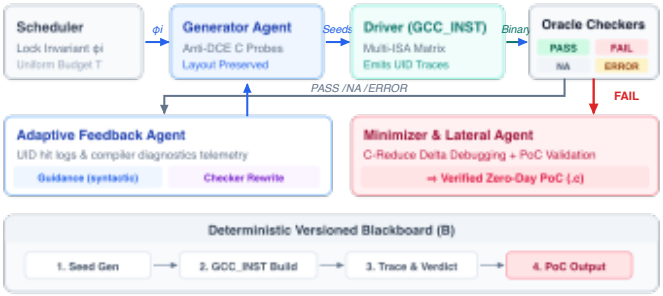


Fig. 5. Online closed-loop agentic fuzzing workflow. A deterministic orchestrator schedules invariant targets under fixed budgets. A generator agent synthesizes anti-DCE seeds, which are built by `GCC_INST` and evaluated by the oracle. Failures trigger C-Reduce minimization and lateral amplification, while non-violations feed telemetry into guidance and checker evolution loops.

D. Agentic Directed Fuzzing

Finally, with the invariant catalog, mapped compiler branches, and programmatic oracles in place, DeFuzz runs its online directed fuzzing loop (Figure 5). Guided by the mapped compiler locations, specialized agents synthesize and mutate program seeds to hit targeted defense branches, while programmatic oracles evaluate every candidate binary.

1) Deterministic Orchestration Pipeline: Rather than letting LLM agents run freely, DeFuzz controls the fuzzing workflow with a deterministic orchestrator. At campaign startup, the orchestrator loads target invariants from the catalog Φ into a queue Q. It locks one invariant ϕ_i at a time along with its ISA-specific checkers, assigning each target an equal time and execution budget T.

Every fuzzing iteration follows a fixed sequence: seed synthesis $\rightarrow$ compilation with `GCC_INST` $\rightarrow$ UID branch trace collection $\rightarrow$ oracle evaluation $\rightarrow$ feedback dispatch. All intermediate states—including generated seeds, compiler logs, and checker verdicts—are recorded on a versioned blackboard $\mathcal{B}$. Pinning a blackboard version deterministically reproduces every test input, compiler trace, and oracle result. Algorithm 1 summarizes this control flow.

2) Checker-Aware Seed Synthesis and DCE Evasion: Random program generators rarely trigger deep compiler defense bugs because their generated code is often removed by early optimizations. In particular, Dead Code Elimination (DCE) strips away unused buffer allocations or redundant pointer writes before backend defense passes run.

To survive optimization and hit targeted Level-3 defense branches, our Generator Agent synthesizes C/C++ probe seeds conditioned on the invariant ϕ_i, the target ISA, and the mapped compiler conditions (§III-B). The agent embeds three anti-DCE patterns into generated seeds:

- Variable-length arrays (VLAs) whose sizes depend on runtime inputs such as `argc`.
- Escaping buffer pointers marked with `volatile` qualifiers or `asm volatile` memory barriers.

Algorithm 1 Online Closed-Loop Agentic Fuzzing Pipeline

Require: Invariant queue Q, Budget T, Instrumented driver `GCC_INST`, Oracle suite $\mathcal{O}$

Initialize versioned blackboard $\mathcal{B}$, Historical seed pool $\mathcal{H} \leftarrow \emptyset$
for target invariant $\phi_i \in Q$ **do**
  Set round budget $t \leftarrow T$
  while $t > 0$ **do**
    $\mathcal{B}$.seed $\leftarrow$ GeneratorAgent($\phi_i, \mathcal{B}$.feedback)
    $\mathcal{H} \leftarrow \mathcal{H} \cup \{\mathcal{B}.\text{seed}\}$
    b, trace $\leftarrow$ Compile(`GCC_INST`, $\mathcal{B}$.seed, ϕ_i.flags)
    $v \leftarrow \mathcal{O}(b, \phi_i.\text{ISA}, \phi_i)$
    if $v ==$ FAIL **then**
      $\mathcal{B}$.poc $\leftarrow$ MinimizerAgent($\mathcal{B}$.seed, $\mathcal{O}, \phi_i$)
      yield ConfirmedBugFound($\mathcal{B}$.poc, ϕ_i)
      LateralDivergence($\phi_i, \mathcal{B}$.seed)
      break
    else if $v \in \{\text{PASS}, \text{NA}, \text{ERROR}\}$ **then**
      $\mathcal{B}$.feedback $\leftarrow$ FeedbackAgent(v, trace, $\mathcal{B}$.seed)
      if $\mathcal{B}$.feedback.rewrite_votes $\geq \tau$ **then**
        ϕ_i.checker $\leftarrow$ TriggerCheckerRewrite(ϕ_i.checker)
        ReplaySeedPool($\mathcal{H}, \phi_i$.checker, `GCC_INST`)
      end if
    end if
    $t \leftarrow t - 1$
  end while
end for

- External or non-inlinable function calls that force the compiler to preserve full stack frames and defense checks.

3) Adaptive Telemetry Feedback and Checker Evolution: When the oracle returns a non-bug verdict (PASS, NA, or ERROR), the Feedback Agent inspects the compiler diagnostics, UID branch hit counts, and checker traces from `GCC_INST`. It then drives two closed-loop adaptation mechanisms:

- **Mutation Guidance Loop:** If the UID trace shows that the seed missed a required compiler branch (for example, `has_vla` evaluated to false), the Feedback Agent produces concrete mutation hints telling the Generator Agent which control-flow or variable types to adjust in the next round.
- **Checker Evolution Loop:** If multiple rounds reveal an incomplete checker rule (reaching a vote threshold τ), DeFuzz updates the ISA checker and immediately replays all historical seeds in $\mathcal{H}$ to catch any previously missed violations.

4) Automated Bug Minimization and Lateral Amplification: When a checker outputs FAIL, the oracle confirms a silent defense bug based solely on deterministic machine evidence. DeFuzz then passes the failing seed to a Minimizer Agent, which pairs C-Reduce with oracle verification. Every reduced candidate is recompiled with `GCC_INST` and rechecked by $\mathcal{O}$ before acceptance. This step strips away unrelated code while preserving the exact memory layout or optimization pattern that triggers the bug, producing a minimal Proof-of-Concept (PoC).

a) Lateral Bug Amplification: Finding a bug does not stop testing for that invariant. Instead, the orchestrator starts a lateral amplification loop, instructing the Generator Agent to mutate surrounding syntax—such as data types, struct nesting, and stack alignment attributes—around the failing PoC. This targeted exploration checks whether the same root cause breaks sibling optimization passes or other backend architectures.

IV. Implementation

The DeFuzz architecture is decoupled into a high-performance evaluation core and a Python-based agentic orchestrator. The core service, implemented in Go, provides deterministic build execution, coverage tracking, and oracle adjudication. It exposes these capabilities to the orchestrator via gRPC for high-throughput programmatic access, and via the Model Context Protocol (MCP) to provide agents with read-only tools for source code search, invariant querying, and delta debugging (C-Reduce) invocation. Crucially, the oracle definitions, static/dynamic checkers, and ISA applicability rules are centralized within the Go core, serving as the single source of truth for all metadata.

The orchestrator governs the explicit agentic loop, maintaining the strict topological order of generation, routing, build, and oracle adjudication. It manages the versioned blackboard, which tracks the lineage of seed corpora, cumulative coverage metrics, and feedback guidance. To ensure rigorous auditability, every execution persists a dedicated artifact bundle containing a serialized state chain, full toolchain configurations, and LLM hyperparameters. This design enables deterministic replay and post-hoc attribution of any confirmed bug claim.

References

[1] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2011.

[2] V. Livinskii, D. Babokin, and J. Regehr, "Random testing for C and C++ compilers with YARPGen," in *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2020.

[3] Y. Liu, Y. Xue, D. Wu, Y. Sun, Y. Li, M. Shi, and Y. Liu, "PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025.

[4] Y. Su, T. Bu, Q. Yang, Y. Ci, and E. Tian, "CIll: CTI-guided invariant generation via LLMs for model checking," arXiv preprint arXiv:2602.23389, 2026. [Online]. Available: https://arxiv.org/abs/2602.23389

[5] T. Hebb, "CVE-2023-4039: GCC's -fstack-protector fails to guard dynamic stack allocations on ARM64," https://rtx.meta.security/mitigation/2023/09/12/CVE-2023-4039.html, 2023.

[6] GCC Bugzilla, "Bug 96191: aarch64 stack_protect_test canary leak," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=96191, 2020.

[7] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.

[8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.